

builds a dictionary ❷. The first scene object in the list of scenes is used as the starting scene ❸.

The last part of the `__init__()` method does some interesting work. The scene manager holds a reference to every scene, so it can send messages to any and every scene. But every scene also needs to be able to send messages to the scene manager. To allow each scene to do that, the last `for` loop in the `__init__()` method calls the special method `_setRefToSceneMgr()` ❹ that lives in the base class of every scene, and it passes `self`, which is a reference to the scene manager. The entire code of this method consists of a single line:

```
def _setRefToSceneMgr(self, oSceneMgr):  
    --- snip ---  
    self.oSceneMgr = oSceneMgr
```

This method just stores this reference back to the scene manager in an instance variable, `self.oSceneMgr`. Each scene can use this variable to make calls to the scene manager. I'll show how scenes use this a little later in this section.

run() Method

For every project you build, you have to write a small main program that instantiates the scene manager. The last step in your main program is a call to the `run()` method of the scene manager. This is where the main loop of the whole program lives. [Listing 15-7](#) contains the code of that method.

```
def run(self):  
  
    --- snip ---  
    clock = pygame.time.Clock()  
  
    # 6 - Loop forever  
    while True:  
  
        ❶ keysDownList = pygame.key.get_pressed()  
  
        # 7 - Check for and handle events  
        ❷ eventsList = []
```